Dart Assignment -1

Name: Hansal Kaushangkumar Anjaria
Enrollment no:202500819010137

A-1

```
void main() {
  for (int i = 2000; i <= 3200; i++) {
    if (i % 7 == 0 && i % 5 != 0) {
      print(i);
    }
  }
}
```

A-2

```
void main() {
  int num = 29;
  bool isPrime = true;

  if (num <= 1) isPrime = false;
  for (int i = 2; i <= num ~/ 2; i++) {
    if (num % i == 0) {
      isPrime = false;
      break;
    }
  }

  print(isPrime ? "$num is prime" : "$num is not prime");
}
```

A-3

```
void main() {
  String input = "hello world! 123";
  int letters = 0, digits = 0;

  for (int i = 0; i < input.length; i++) {
    if (input[i].toLowerCase().compareTo('a') >= 0 &&
        input[i].toLowerCase().compareTo('z') <= 0) {
      letters++;
    } else if (input[i].compareTo('0') >= 0 && input[i].compareTo('9') <= 0) {
      digits++;
    }
  }

  print("LETTERS $letters DIGITS $digits");
}
```

A-4

```
void main() {
  int start = 1, end = 30;
  for (int i = start; i <= end; i++) {
    if (i % 2 == 0) {
      print("Square of $i = ${i * i}");
    }
  }
}
```

A-5

```
class AgeException implements Exception {
  String message;
  AgeException(this.message);
}

void main() {
  try {
    print("Enter your age:");
    String? input = stdin.readLineSync();
    int age = int.parse(input!);
    if (age < 18) throw AgeException("Age must be 18 or above");
    print("Age entered: $age");
  } on FormatException {
    print("Invalid input. Please enter a valid number.");
  } on AgeException catch (e) {
    print(e.message);
  }
}
```

A-6

```
class Task {
  String title;
  bool isCompleted;

  Task(this.title, [this.isCompleted = false]);
}

void main() {
  List<Task> tasks = [];

  void addTask(String title) => tasks.add(Task(title));
  void removeTask(int index) => tasks.removeAt(index);
  void markCompleted(int index) => tasks[index].isCompleted = true;

  void displayTasks() {
    print("Pending:");
    tasks.where((t) => !t.isCompleted).forEach((t) => print("- ${t.title}"));
    print("Completed:");
    tasks.where((t) => t.isCompleted).forEach((t) => print("- ${t.title}"));
  }

  // Example usage
  addTask("Learn Dart");
  addTask("Complete Assignment");
  markCompleted(0);
  displayTasks();
}
```

A-7

```
class BankAccount {
  double _balance = 0.0;

  void deposit(double amount) {
    if (amount > 0) _balance += amount;
  }

  void withdraw(double amount) {
    if (amount > _balance) {
      print("Insufficient balance.");
    } else {
      _balance -= amount;
    }
```

```
  }

  void checkBalance() => print("Current Balance: $_balance");
}

void main() {
  BankAccount acc = BankAccount();
  acc.deposit(1000);
  acc.withdraw(200);
  acc.checkBalance();
}
```

A-8

```
class Employee {
  double calculateSalary() => 0.0;
}

class PermanentEmployee extends Employee {
  double baseSalary;
  double bonus;

  PermanentEmployee(this.baseSalary, this.bonus);

  @override
  double calculateSalary() => baseSalary + bonus;
}

class ContractEmployee extends Employee {
  double hourlyRate;
  int hoursWorked;

  ContractEmployee(this.hourlyRate, this.hoursWorked);

  @override
  double calculateSalary() => hourlyRate * hoursWorked;
}

void main() {
  Employee emp1 = PermanentEmployee(50000, 5000);
  Employee emp2 = ContractEmployee(500, 40);

  print("Permanent Employee Salary: ${emp1.calculateSalary()}");
  print("Contract Employee Salary: ${emp2.calculateSalary()}");
}
```

Code Analysis Tasks

A-1

Given Code Issue: Constructor parameters are incorrectly assigned.

```
class Student {
  int id;
  String name;

  Student(int id, String name) {
    this.id = id;
    this.name = name;
  }
```

```
  void display() {
    print("ID: $id, Name: $name");
  }
}

void main() {
  Student s = Student(1, "Rahul");
  s.display();
}
```

A-2

Issue: Missing curly braces and logical error in grading.

```
import 'dart:io';

void main() {
  print("Enter marks:");
  int marks = int.parse(stdin.readLineSync()!);

  if (marks > 90) {
    print("A");
  } else if (marks > 75) {
    print("B");
  } else if (marks > 60) {
    print("C");
  } else {
    print("D");
  }
}
```

A-3

Issue: Negative price allowed.

```
class Product {
  String name;
  double price;

  Product(this.name, double price) {
    if (price < 0) throw ArgumentError("Price cannot be negative");
    this.price = price > 30000 ? price * 0.9 : price;
  }
}

void main() {
  try {
    Product p = Product("Laptop", 50000);
    print(p.price);
  } catch (e) {
    print(e);
  }
}
```

A-4

Given Output: false because == compares references by default.

```
class Person {
  String name;
  Person(this.name);
```

```
  @override
  bool operator ==(Object other) =>
      identical(this, other) || other is Person && name == other.name;

  @override
  int get hashCode => name.hashCode;
}

void main() {
  Person p1 = Person("Amit");
  Person p2 = Person("Amit");
  print(p1 == p2); // true
}
```

A-5

Issue: Missing break in first case.

```
void main() {
  int day = 1;

  switch (day) {
    case 1:
      print("Monday");
      break;
    case 2:
      print("Tuesday");
      break;
    case 3:
      print("Wednesday");
      break;
    case 4:
      print("Thursday");
      break;
    case 5:
      print("Friday");
      break;
    case 6:
      print("Saturday");
      break;
    case 7:
      print("Sunday");
      break;
    default:
      print("Invalid day");
  }
}
```